Operating System

A Complete and Detailed Study Guide

Definition - Need - Functions - Categories - Types - Why each OS was introduced - History - Explanation of the lecture slides

Contents

1. What is an Operating System?
2. Why do we need an Operating System?
3. Functions of an Operating System
4. Categories of Operating Systems
5. Types (Examples) of Operating Systems
6. Why each OS was introduced, one by one
7. History of Operating Systems
8. Explanation of the given lecture images
9. Quick revision summary

1. What is an Operating System?

A computer is made of **hardware** (physical parts such as the processor, memory, hard disk, keyboard, mouse, monitor and printer) and **software** (programs that tell the hardware what to do). Software is of two broad kinds: **application software** (MS Word, a web browser, a game) and **system software** (software that runs the computer itself). The **Operating System (OS)** is the most important piece of system software.

Definition: An operating system is a collection of programs that acts as an **interface (middleman) between the user and the computer hardware**. It manages all hardware and software resources, and provides a platform on which application programs can run.

A simple analogy: think of a computer as a large restaurant. The hardware is the kitchen equipment, the applications are the chefs, and the users are the customers. The OS is the **manager**: it takes orders from customers, decides which chef cooks what and when, shares the equipment between chefs, keeps the kitchen safe and tidy, and brings the food back to the customer. Without the manager, everything is chaos.

Key points about an OS

- It is the **first program loaded** when the computer starts (a process called booting) and it keeps running until the computer is shut down.
- It acts as a **resource manager**: CPU, memory, storage and I/O devices are shared among many programs, and the OS decides who gets what.
- It acts as a **translator**: it converts the user's instructions into machine language (0s and 1s) the hardware understands, and converts the hardware's response back into a form the user understands.
- It acts as a **control program**: it prevents errors and improper use of the computer.
- It is found in almost every computing device: PCs, laptops, servers, smartphones, tablets, smart TVs, ATMs, cars, routers and even washing machines.

Parts of an OS

- **Kernel** - the core. It always stays in memory and directly controls CPU, memory and devices.
- **System programs / utilities** - tools such as file managers, disk cleaners and task managers.
- **Shell / User interface** - the part the user interacts with, either a command line (CUI) or graphical (GUI). This is explained further in Section 8.

2. Why do we need an Operating System?

Imagine a computer without an OS. Every programmer would have to write code to talk directly to the disk, the keyboard and the screen, in raw machine language, for every single program. That is slow, error-prone and impossible for ordinary people. The OS exists to solve this problem. The main reasons are:

Need	Explanation
Convenience / ease of use	Users click an icon or type a short command instead of writing machine code. The OS hides the complex hardware details.
Efficient use of hardware	The CPU is very fast and would sit idle while waiting for slow devices. The OS keeps the CPU busy by switching between programs, giving high utilisation.
Resource management	Many programs want the same CPU, memory and printer at the same time. The OS allocates and de-allocates resources fairly and avoids conflicts.
Running multiple programs	You can listen to music, browse the web and edit a document together. The OS makes this multitasking possible.
Hardware independence	A program such as a browser can run on many different computers because the OS handles hardware differences through device drivers.
File and data management	The OS organises data into files and folders, allows creating, copying, deleting and searching, and stores them safely on disk.
Security and protection	It stops unauthorised users and faulty programs from accessing or damaging data (passwords, user accounts, permissions).
Error handling	If a program crashes or a device fails, the OS detects it and prevents the whole system from collapsing.
Communication	It provides networking so computers can share files, printers and the internet.
Platform for applications	Developers write apps for an OS (using its system calls and APIs) instead of for raw hardware.

3. Functions of an Operating System

The lecture slides list six main functions. Each one is explained below in detail (see also Section 8, Image 3).

3.1 Memory Management

The OS keeps a record of the primary memory (RAM): which parts are currently in use, by which program, and which parts are free. When a program or process requests memory, the OS allocates it; when the program finishes, the OS takes the memory back. Techniques used include contiguous allocation, paging, segmentation and virtual memory (using disk space as an extension of RAM).

3.2 Processor Management (CPU Scheduling)

Many processes want the CPU but there is usually only one (or a few) processor cores. The OS decides which process runs next and for how long (scheduling), allocates the CPU to it and takes the CPU back when it is no longer required or its time is over. Common algorithms are First Come First Served, Shortest Job First, Round Robin and Priority scheduling. The part that does this is called the scheduler, and switching from one process to another is a context switch.

3.3 Device Management

The OS keeps track of every device (keyboard, mouse, printer, disk, monitor). The module that does this is called the **I/O controller**. It decides which process gets a device, when, and for how much time. It uses **device drivers** (small programs that know how to talk to a particular device), buffering, caching and spooling (for example a print queue).

3.4 File Management

The OS manages files and directories: creating, naming, opening, reading, writing, moving and deleting. It tracks where files are stored on disk (file systems such as FAT32, NTFS, ext4, APFS), controls who may read or modify a file (permissions), and, as the slide says, allocates and de-allocates resources and decides who gets the resources.

3.5 Security

The OS prevents unauthorised access to programs and data by means of passwords and similar techniques such as user accounts, biometric login, file permissions, encryption and firewalls.

3.6 Control over System Performance

The OS records the delays between a request for a service and the response from the system. It monitors usage of CPU, memory and disk, produces logs and statistics, and helps administrators tune the system (for example the Task Manager in Windows or the top command in Linux).

Other functions

- **Job accounting** - tracking time and resources used by users and jobs.
- **Error detection and recovery** - handling hardware faults and program errors.
- **Coordination** between compilers, interpreters, assemblers and other software and users.

- **User interface** - providing CUI or GUI so people can use the computer.
- **Networking** - supporting communication between systems.

4. Categories of Operating Systems

Operating systems can be grouped in several ways. The word *category* here means classification by how the OS works or who it serves. (The word *type* in the lecture slide refers to named products such as Windows or Linux, covered in Section 5.)

4.1 By number of users

- **Single-user OS** - supports one user at a time (MS-DOS, early Windows, a personal phone OS).
- **Multi-user OS** - several users can use the system at the same time with separate accounts, usually via terminals or network (UNIX, Linux, Windows Server).

4.2 By number of tasks

- **Single-tasking** - runs only one program at a time (MS-DOS).
- **Multitasking** - runs many programs at once by rapidly switching the CPU (Windows, Linux, macOS, Android).

4.3 By the way they process work

Category	How it works	Examples / Uses
Batch OS	Similar jobs are collected into batches and run one after another without user interaction. Good throughput, but no interaction and long waiting.	IBM mainframes of the 1950s-60s, payroll and bill processing.
Multiprogramming OS	Several jobs are kept in memory; when one waits for I/O, the CPU switches to another, so the CPU is rarely idle.	OS/360, early UNIX.
Time-sharing (multitasking) OS	The CPU time is divided into small slices (quanta) given to each user or program in turn, so each user feels they have the whole computer. Gives fast response.	UNIX, Linux, Windows.
Real-time OS (RTOS)	Must respond within a strict deadline. Hard real-time: missing a deadline is a failure (air-bag, pacemaker). Soft real-time: occasional delay tolerated (video streaming).	VxWorks, QNX, FreeRTOS.
Distributed OS	Many independent computers connected by a network appear to the user as one single system. Shares load and resources.	Amoeba, LOCUS, cluster systems.
Network OS	Runs on a server and provides shared files, printers, users and security to client computers on a network.	Windows Server, Novell NetWare, Linux servers.
Embedded OS	Small, specialised OS built into a device with limited memory and a fixed purpose.	Smart TV, ATM, washing machine, router, car systems.
Mobile OS	Designed for touch-screen, battery-powered devices, with sensors, app stores and wireless connectivity.	Android, iOS.

4.4 By user interface (see Image 5)

- **CUI / CLI (Character or Command-line User Interface)** - user types text commands (DOS, Linux terminal).
- **GUI (Graphical User Interface)** - user uses windows, icons, menus and a pointer (Windows, macOS, Android).

4.5 By source code license

- **Proprietary / closed-source** - owned by a company, source code hidden (Windows, macOS, iOS).
- **Open-source** - source code is public and can be modified and shared (Linux, Android Open Source Project).

5. Types of Operating Systems (the list in Image 4)

The slide lists eight well-known operating systems: DOS, Windows, Linux, Mac, Solaris, iOS, OS/2 and Android. Here is each in detail.

OS	Developer	First release	Interface	Main use / features
DOS (MS-DOS)	Microsoft (for IBM PC)	1981	CUI (command line)	Single-user, single-tasking, 16-bit. Commands like dir, copy, del. Ran on the original IBM PC.
Windows	Microsoft	1985 (Windows 1.0)	GUI	Most used desktop OS. Multitasking, plug-and-play, huge software and game support, NTFS. Latest: Windows 11 (2021).
Linux	Linus Torvalds and community	1991	CLI and GUI	Free and open-source, UNIX-like, stable, secure, customisable. Runs servers, supercomputers, embedded devices and Android's kernel.
Mac (macOS)	Apple	1984 (System 1); Mac OS X 2001	GUI	UNIX-based desktop OS for Apple Macs. Known for design, stability, creative software.
Solaris	Sun Microsystems (now Oracle)	1992 (Solaris 2; SunOS since 1982)	CLI and GUI	UNIX OS for enterprise servers and workstations. Strong scalability, ZFS file system, DTrace.
iOS	Apple	2007 (iPhone OS; renamed iOS 2010)	Touch GUI	Mobile OS for iPhone and iPad. Closed, secure, App Store ecosystem.
OS/2	IBM and Microsoft (later IBM only)	1987	CUI, later GUI (Presentation Manager)	32-bit multitasking OS for PCs and banks/ATMs. Support ended in 2006.
Android	Android Inc., then Google	2008 (first phone)	Touch GUI	Open-source mobile OS built on the Linux kernel. World's most used smartphone OS.

Note: the slide writes "Mac" and "IOS". Mac means Apple's macOS (earlier Mac OS) and IOS means Apple's mobile OS iOS. It should not be confused with Cisco's IOS, which is the router operating system.

In detail

DOS. Disk Operating System. A text-only system where the user must remember and type commands. It could run only one program at a time and had no graphics. It is the foundation on which early Windows ran.

Windows. Started as a graphical shell on top of DOS and later became a complete OS (Windows NT family). Versions: 1.0, 3.0/3.1, 95, 98, ME, NT, 2000, XP, Vista, 7, 8, 10, 11.

Linux. Strictly Linux is the kernel; combined with GNU tools it forms distributions (distros) such as Ubuntu, Debian, Fedora, Red Hat and Arch.

macOS. Descended from NeXTSTEP and BSD UNIX. Named Mac OS X from 2001, OS X from 2012 and macOS from 2016.

Solaris. Used in banks, telecom and large data centres; its descendant projects include illumos and OpenIndiana.

iOS. Derived from macOS (Darwin). Features multi-touch, strict app sandboxing and regular updates.

OS/2. Offered true pre-emptive multitasking before Windows 95 did, but lost the market to Windows.

Android. Used by phones, tablets, TVs (Android TV), watches (Wear OS) and cars (Android Auto).

6. Why Each Operating System Was Introduced, One by One

Each OS appeared to solve a specific problem of its time, or to serve a new kind of hardware or user.

OS	Problem or gap at that time	Why it was introduced
Early batch systems (GM-NAA I/O, 1956)	Operators loaded programs by hand; the expensive CPU sat idle between jobs.	To automatically run one job after another and reduce wasted machine time.
Multiprogramming / OS/360 (1964)	The CPU idled while waiting for slow I/O. Every IBM computer model needed different software.	To keep many jobs in memory and use the CPU fully; one OS for a whole family of machines.
Time-sharing (CTSS 1961, Multics 1965)	Batch users waited hours for results and could not interact.	To let many users work interactively at the same time with quick response.
UNIX (1969)	Multics was big and complex.	To build a simple, small, portable multi-user OS. Written in C (1973) so it could move between machines.
DOS (1981)	The new IBM PC needed an OS for a cheap, single-user personal computer.	To give personal computers a simple, low-cost OS.
Mac OS (1984)	DOS needed typed commands, which ordinary people found hard.	To bring the graphical, mouse-driven interface (pioneered at Xerox PARC) to everyday users.
Windows (1985)	DOS users wanted graphics and multitasking, and Microsoft wanted its own GUI.	To add a GUI on top of DOS and later grow into a full OS (Windows NT, XP, 7, 10, 11).
OS/2 (1987)	DOS could use only a small amount of memory and had no true multitasking. IBM wanted a PC OS it controlled.	To be the 32-bit, multitasking successor to DOS.
Linux (1991)	UNIX was costly and licence-restricted; Minix was only for teaching.	Torvalds wanted a free UNIX-like kernel for PCs. Open for everyone to improve.
Solaris (1992)	Sun needed a powerful, scalable OS for its servers and multiprocessor workstations.	To give enterprises a reliable, high-performance UNIX (SunOS evolved into Solaris).
iOS (2007)	Phones had clumsy keypads and limited operating systems.	To power the iPhone with a touch-first, app-based mobile OS.
Android (2008)	iOS was limited to Apple devices; phone makers needed a free OS.	To provide an open, free mobile OS that any manufacturer can use.

7. History of Operating Systems

The history is usually divided into generations that follow the generations of computer hardware.

First generation (1940s - early 1950s): No operating system

Machines such as ENIAC (1945) used vacuum tubes and were programmed directly with switches, plug-boards and later punched cards. One programmer used the whole machine at a time, loaded the program by hand and watched the lights for results. There was no OS at all, and much time was wasted in setup.

Second generation (1950s - mid 1960s): Batch systems

With transistors, computers became more reliable. The first simple OS, **GM-NAA I/O** (1956), was created by General Motors and North American Aviation for the IBM 704 to run jobs in a batch automatically. Programs were written in FORTRAN or assembly on punched cards, collected by an operator, run, and results printed. Monitors and early job-control languages appeared. The Atlas computer (1962) introduced virtual memory.

Third generation (mid 1960s - 1970s): Multiprogramming and time-sharing

Integrated circuits made machines faster. Key events:

- **CTSS (1961)** at MIT - first major time-sharing system.
- **OS/360 (1964)** by IBM - one OS for a whole family of computers, with multiprogramming.
- **Multics (1964-65)** by MIT, Bell Labs and GE - ambitious time-sharing OS that inspired many ideas.
- **UNIX (1969-1971)** by Ken Thompson and Dennis Ritchie at Bell Labs; rewritten in C in 1973. It introduced a hierarchical file system, pipes and the shell, and became the ancestor of Linux, macOS, Solaris and others.

Fourth generation (1970s - 1990s): Personal computers and GUIs

Microprocessors made personal computers affordable. **CP/M** (1974) was an early PC OS. **MS-DOS** (1981) came with the IBM PC. The Xerox Alto (1973) pioneered the graphical interface, which Apple brought to the public with the **Macintosh (1984)**. Microsoft released **Windows 1.0 (1985)**, and Windows 3.0 (1990) became very popular. **OS/2** appeared in 1987. Richard Stallman began the GNU project (1983), and **Linux** was announced by Linus Torvalds in 1991. **Solaris 2** appeared in 1992. **Windows NT (1993)** and **Windows 95** brought 32-bit, plug-and-play and the Start menu.

Fifth generation (2000s - present): Internet, mobile and cloud

Windows XP (2001) and Mac OS X (2001) unified home and professional use. Linux dominated servers and supercomputers. **iOS (2007)** and **Android (2008)** started the smartphone era. Cloud computing, virtualisation (VMware, Hyper-V, KVM, Docker containers), Chrome OS, Windows 10 (2015) and Windows 11 (2021) followed. Today OS design focuses on security, battery efficiency, AI features, cloud sync and connected devices (IoT).

Timeline summary

Year	Event
1945	ENIAC - no OS, manual programming
1956	GM-NAA I/O - first batch OS
1961	CTSS - time-sharing
1962	Atlas - virtual memory
1964	IBM OS/360
1965	Multics
1969-73	UNIX created and rewritten in C
1974	CP/M
1981	MS-DOS with IBM PC
1984	Apple Macintosh System 1 (GUI)
1985	Windows 1.0
1987	OS/2
1991	Linux kernel announced
1992	Solaris 2
1993-95	Windows NT, Windows 95
2001	Windows XP, Mac OS X
2007	iOS (iPhone)
2008	Android (HTC Dream)
2015 / 2021	Windows 10 / Windows 11

8. Explanation of the Given Lecture Images

8.1 Image 1 - "Operating System" (definition slide)

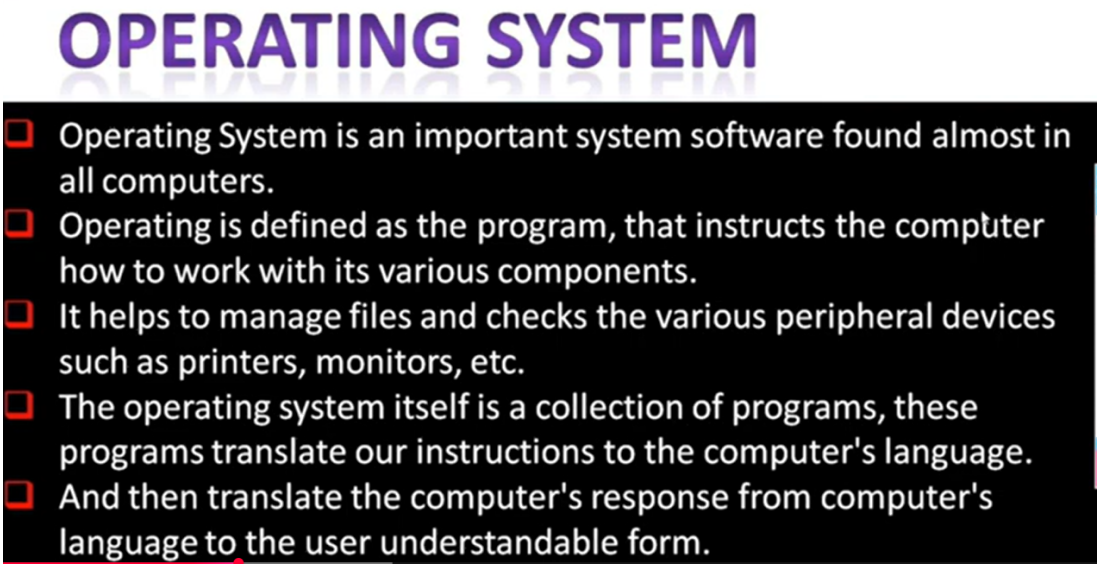


Image 1: Operating System definition slide

This slide gives five basic points that define an OS:

- **"An important system software found almost in all computers."** The OS is system software, not an application. Nearly every computer, phone and smart device has one.
- **"The program that instructs the computer how to work with its various components."** (The slide's word "Operating" means "Operating System".) The OS tells the CPU, memory, disk and other parts how to cooperate.
- **"It helps to manage files and checks the various peripheral devices such as printers, monitors."** This is file management and device management.
- **"The OS is a collection of programs that translate our instructions to the computer's language."** Humans use words, clicks and taps; computers understand only binary. The OS translates in this direction.
- **"Then translate the computer's response to a user-understandable form."** The reverse translation: the binary result is shown on screen as text, pictures or sound.

In short, the OS works like an interpreter standing between a human user and the machine, translating both ways.

8.2 Image 2 - "Characteristics of Operating System" (hub diagram)

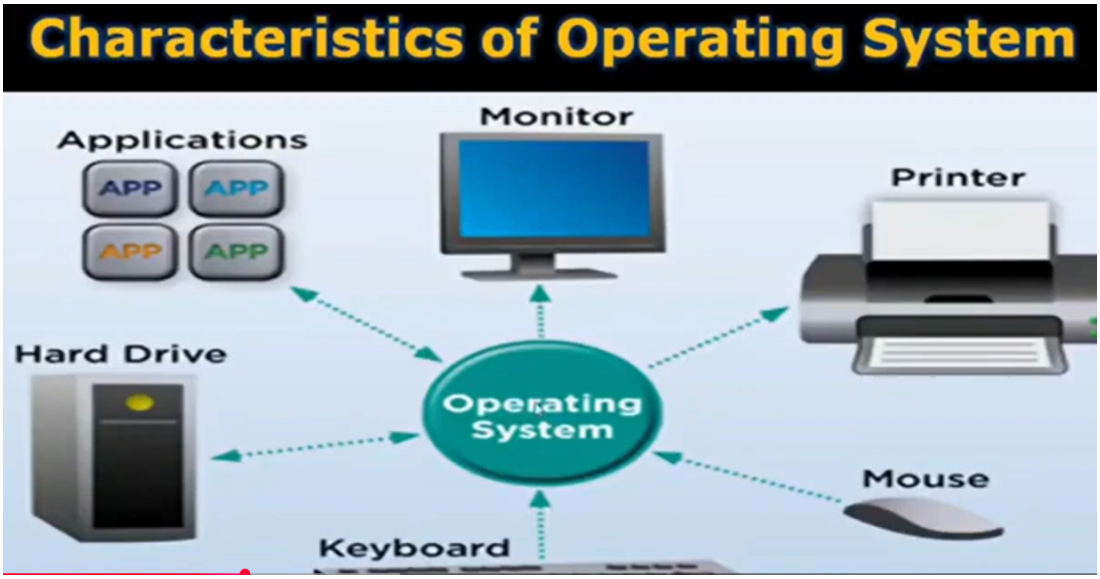
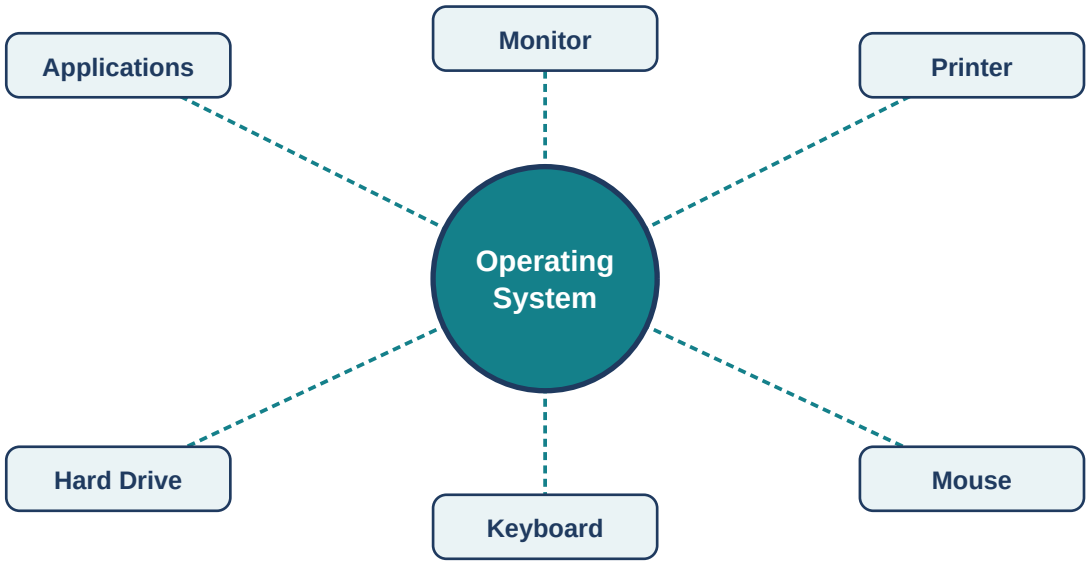


Image 2: Operating System at the centre of hardware and applications

The picture shows the OS as a circle at the **centre**, connected by two-way dotted arrows to Applications, Monitor, Printer, Hard Drive, Keyboard and Mouse. A redrawn version is below.



What it teaches:

- **Central coordinator.** Nothing talks to the hardware directly. Applications and devices communicate through the OS.
- **Input devices** (keyboard, mouse) send data *to* the OS. **Output devices** (monitor, printer) receive data *from* the OS. **Storage** (hard drive) and **applications** exchange data in both directions, which is why those arrows point both ways.
- **Applications** (the four APP icons) request services such as opening a file, drawing a window or printing, and the OS provides them through system calls.
- **Characteristics** shown by this diagram: resource management, device abstraction, mediation between software and hardware, and a single point of control and security.

8.3 Image 3 - Functions of the Operating System

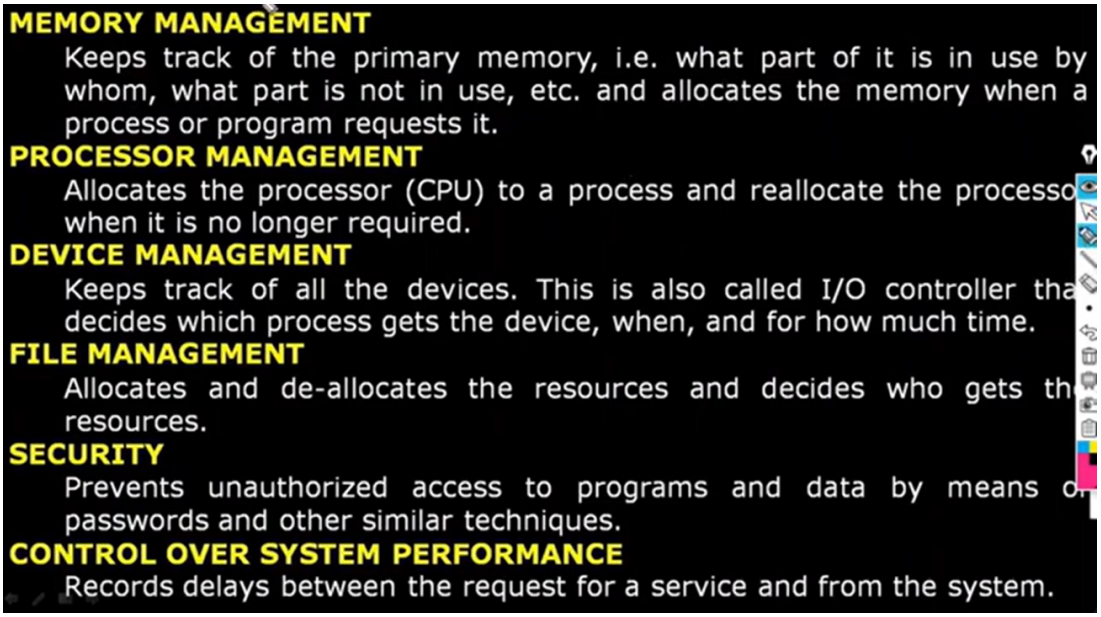


Image 3: Six functions of an operating system

This slide lists the six management functions already explained in Section 3. Summary table:

Function	What the slide says	In simple words
Memory Management	Keeps track of primary memory, what part is in use by whom and what is not, and allocates memory when a process requests it.	The OS is the landlord of RAM: gives space, remembers who has it, takes it back.
Processor Management	Allocates the CPU to a process and reallocates it when no longer required.	Decides which program uses the CPU and when.
Device Management	Tracks all devices (I/O controller); decides which process gets the device, when and for how long.	Traffic police for printers, disks and keyboards.
File Management	Allocates and de-allocates resources and decides who gets them.	Organises, stores and protects files and folders.
Security	Prevents unauthorised access to programs and data using passwords and similar techniques.	Guard that checks who you are.
Control over System Performance	Records delays between the request for a service and the response from the system.	Monitors speed and finds slowdowns.

(The right edge of the screenshot shows the lecturer's pen and annotation toolbar; it is not part of the content.)

8.4 Image 4 - Types of Operating System

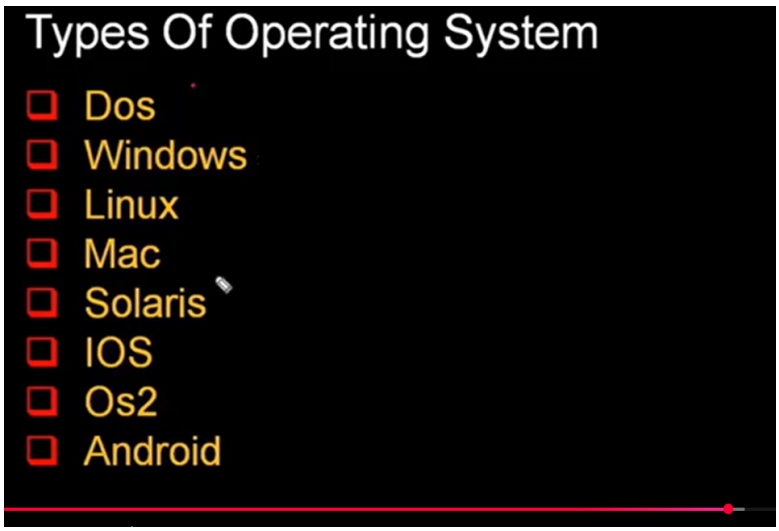


Image 4: List of operating system types

The slide gives eight examples: DOS, Windows, Linux, Mac, Solaris, iOS, OS/2, Android. They are described in Section 5 and the reasons for their introduction in Section 6. A useful way to remember them is by family:

- **Microsoft family:** DOS, Windows.
- **UNIX-like family:** Linux, Mac (macOS), Solaris, iOS (derived from macOS), Android (Linux kernel).
- **IBM / joint:** OS/2.
- **By device:** desktop/server - DOS, Windows, Linux, Mac, Solaris, OS/2; mobile - iOS, Android.

8.5 Image 5 - Hand-drawn diagram: OS, CUI and GUI

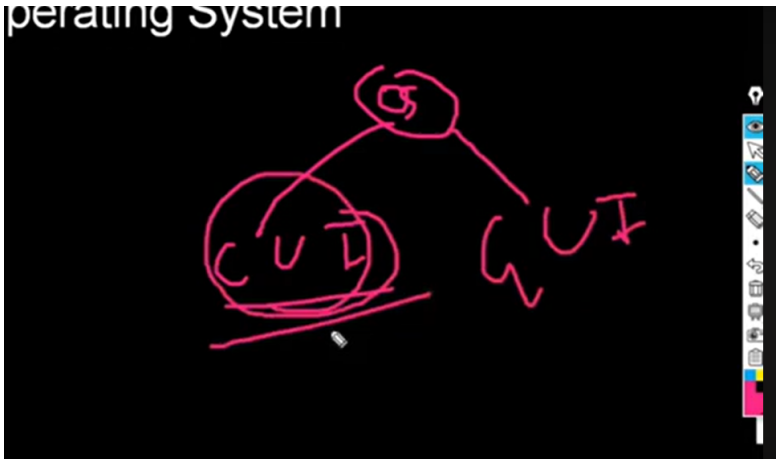
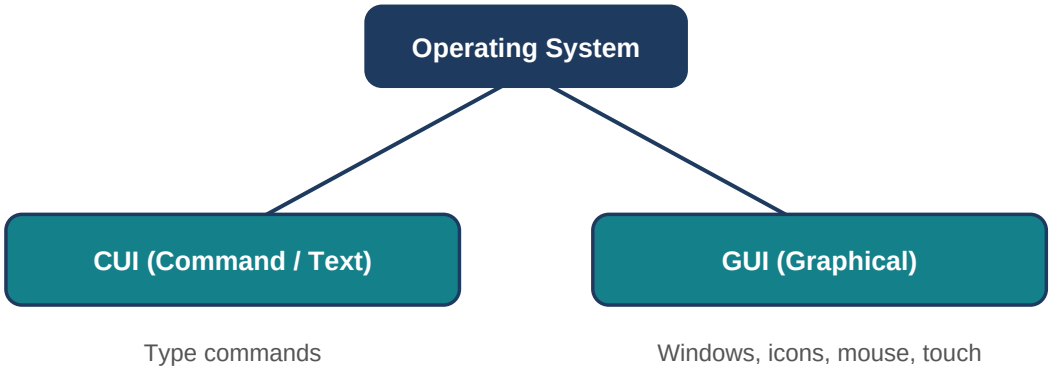


Image 5: Lecturer's hand-drawn sketch (OS branching into CUI and GUI)

The lecturer drew a circle labelled **OS** at the top with two lines going down. The left branch ends in **CUI** (circled and underlined for emphasis) and the right branch ends in **GUI**. This shows that an OS provides **two kinds of user interface**. A cleaned-up version:



Feature	CUI (Command / Character User Interface)	GUI (Graphical User Interface)
Full form	Character (or Command-line) User Interface	Graphical User Interface
How the user works	Types text commands on the keyboard	Clicks icons, menus and windows with a mouse or touch
Ease of use	Harder; commands must be remembered	Easy and intuitive, good for beginners
Speed and resources	Very light, fast, uses little memory and CPU	Needs more memory, CPU and graphics
Power and automation	Excellent for scripting and administration	Less suited to automation
Examples	MS-DOS, Linux terminal, Windows Command Prompt, PowerShell	Windows, macOS, Ubuntu desktop, Android, iOS
Sample action	`copy a.txt d:\` to copy a file	Drag the file icon to another folder

9. Quick Revision Summary

- The **operating system** is system software that sits between user and hardware, manages resources and runs applications.
- It is **needed** for convenience, efficiency, multitasking, file handling, security and hardware independence.
- **Functions:** memory, processor, device and file management, security, and performance control.
- **Categories:** single-user / multi-user, single-task / multitask, batch, multiprogramming, time-sharing, real-time, distributed, network, embedded, mobile; also CUI / GUI and open / closed source.
- **Types:** DOS, Windows, Linux, Mac, Solaris, iOS, OS/2, Android.
- **History:** no OS (1940s), batch (1950s), time-sharing and UNIX (1960s-70s), PC and GUI (1980s-90s), mobile and cloud (2000s onward).
- **Interfaces:** an OS provides CUI (commands) and GUI (graphics).
- **Slide 2's message:** the OS is the central hub that links applications and all hardware devices.

Practice questions

- Define an operating system and explain why it is called an interface.
- List and explain the six functions of an OS.
- Differentiate between batch, time-sharing and real-time operating systems.
- Why was MS-DOS replaced by GUI-based systems?
- Compare CUI and GUI with examples.
- Write short notes on UNIX, Linux and Android.
- Describe the history of operating systems generation by generation.

Note: release years and details above are the commonly accepted dates; some sources differ by a year depending on whether the announcement, development or public release is counted.